

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network (CNN) is a deep learning architecture commonly used for image processing and image classification.

A basic CNN consists of the following layers:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is given by:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n)K(m, n) \quad (1)$$

where:

- X represents the input image.
- K represents the kernel or filter.
- Y represents the resulting feature map.

The output feature-map size is calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1 \quad (2)$$

where:

- N = Input size
- F = Filter size
- P = Padding
- S = Stride

3.1 Common Convolution Kernels

A kernel is a small matrix that slides over an image to extract important visual features such as edges, textures, corners, and smooth regions.

Table 1: Common convolution kernels

Kernel	Size	Purpose
Identity	3×3	Preserve the original image
Sobel-X	3×3	Detect vertical edges
Sobel-Y	3×3	Detect horizontal edges
Laplacian	3×3	Detect edges in all directions
Sharpen	3×3	Enhance fine details
Box Blur	3×3	Smooth the image
Gaussian Blur	3×3	Reduce noise
Emboss	3×3	Create a three-dimensional effect
Outline	3×3	Highlight object boundaries
Motion Blur	5×5	Simulate motion blur

4 Numerical Examples

4.1 Numerical Example 1: Convolution

Consider the input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and the kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The first position is:

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

The second position is:

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

The third position is:

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

The fourth position is:

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Therefore, the feature map is:

$$\boxed{\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}}$$

4.2 Numerical Example 2: Max Pooling

Consider the feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max-pooling filter:

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Therefore:

$$\boxed{\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}}$$

4.3 Numerical Example 3: Parameter Calculation

Suppose the input image has dimensions:

$$32 \times 32 \times 3$$

and the convolution layer contains:

- Number of filters = 16
- Kernel size = 3×3

The number of trainable parameters is:

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 \tag{3}$$

$$= (27 + 1) \times 16 \tag{4}$$

$$= \boxed{448} \tag{5}$$

5 Dataset

The dataset used for this experiment is CIFAR-10.

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10

- Image Size: $32 \times 32 \times 3$

The ten classes are:

Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

6 Experimental Procedure

6.1 Task 1: CIFAR-10 Dataset Visualization

The CIFAR-10 dataset was loaded using TensorFlow/Keras. Ten sample images were displayed and the class distribution of the training dataset was plotted.

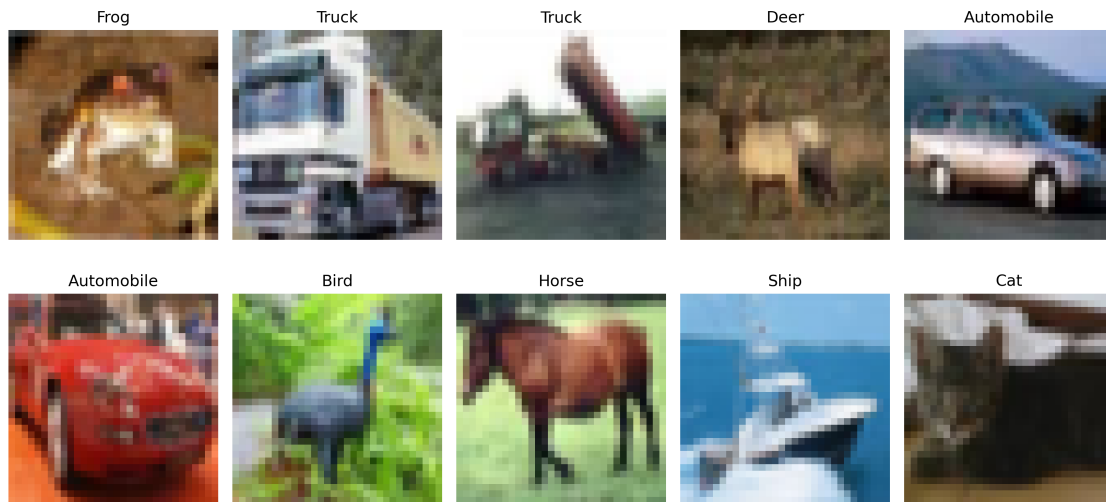


Figure 1: Sample images from the CIFAR-10 dataset

Inference:

The sample images demonstrate the different object categories present in the CIFAR-10 dataset. Each image is a 32×32 RGB image belonging to one of ten classes.

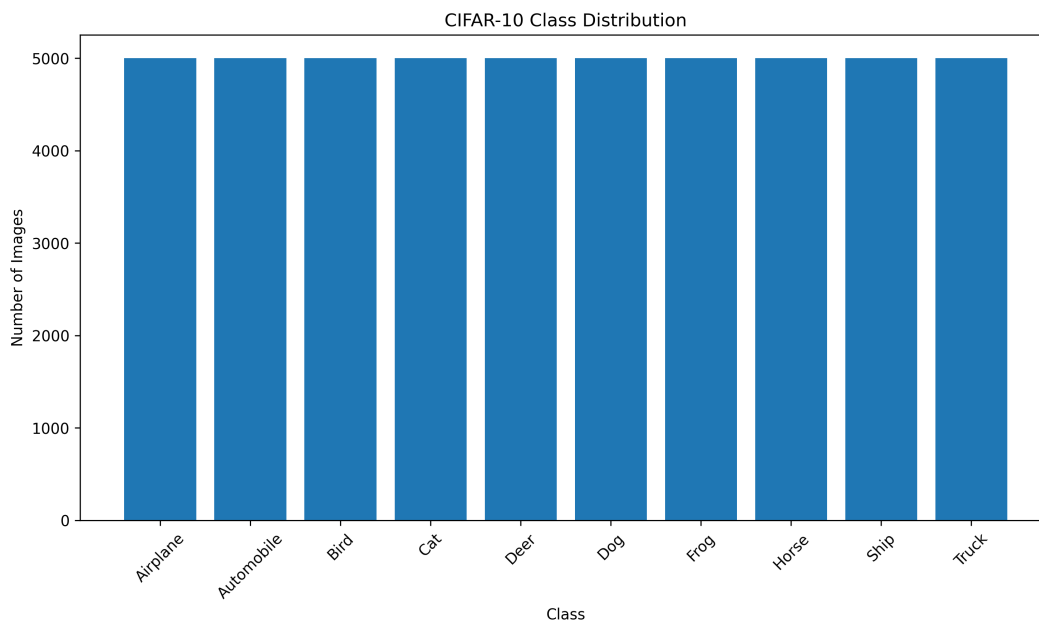


Figure 2: CIFAR-10 class distribution

Inference:

The class distribution is approximately balanced, with around 5,000 training images available for each class. Therefore, no class has a major numerical advantage over the others.

6.2 Task 2: Convolution Kernel Comparison

Three convolution kernel sizes were compared:

- 3×3
- 5×5
- 7×7

Valid padding and stride 1 were used.

Table 2: Feature-map dimensions for different kernel sizes

Kernel Size	Input Size	Output Size
3×3	32×32	30×30
5×5	32×32	28×28
7×7	32×32	26×26

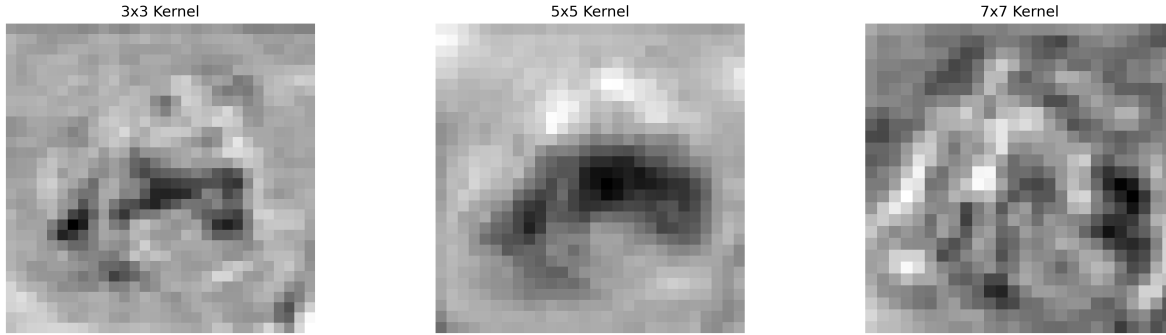


Figure 3: Comparison of convolution feature maps

Inference:

Increasing the kernel size reduces the spatial dimensions of the feature map when valid padding is used. Larger kernels cover a larger receptive field but produce smaller output feature maps.

6.3 Task 3: Stride and Padding

The effects of stride and padding were studied using a 3×3 kernel.

Table 3: Effect of stride and padding

Padding	Stride	Output Size
Valid	1	30×30
Valid	2	15×15
Same	1	32×32

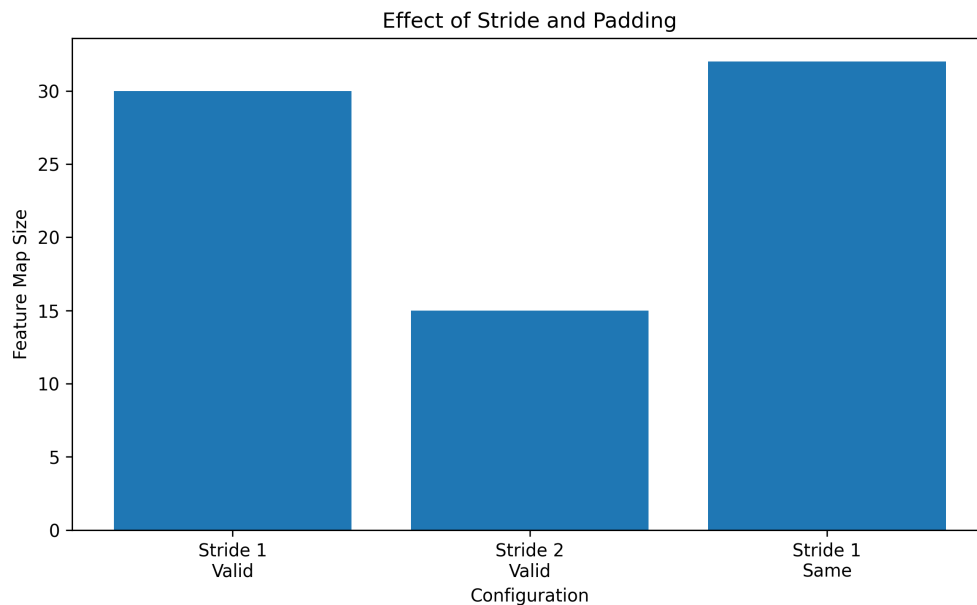


Figure 4: Effect of stride and padding

Inference:

Increasing the stride decreases the spatial dimensions of the output feature map. Same padding preserves the spatial dimensions for stride 1, whereas valid padding reduces the output dimensions.

6.4 Task 4: Feature Map Visualization

A convolution layer containing 16 filters with kernel size 3×3 was applied to an input image.

The resulting feature tensor had dimensions:

$$1 \times 32 \times 32 \times 16$$

Eight feature maps were visualized.

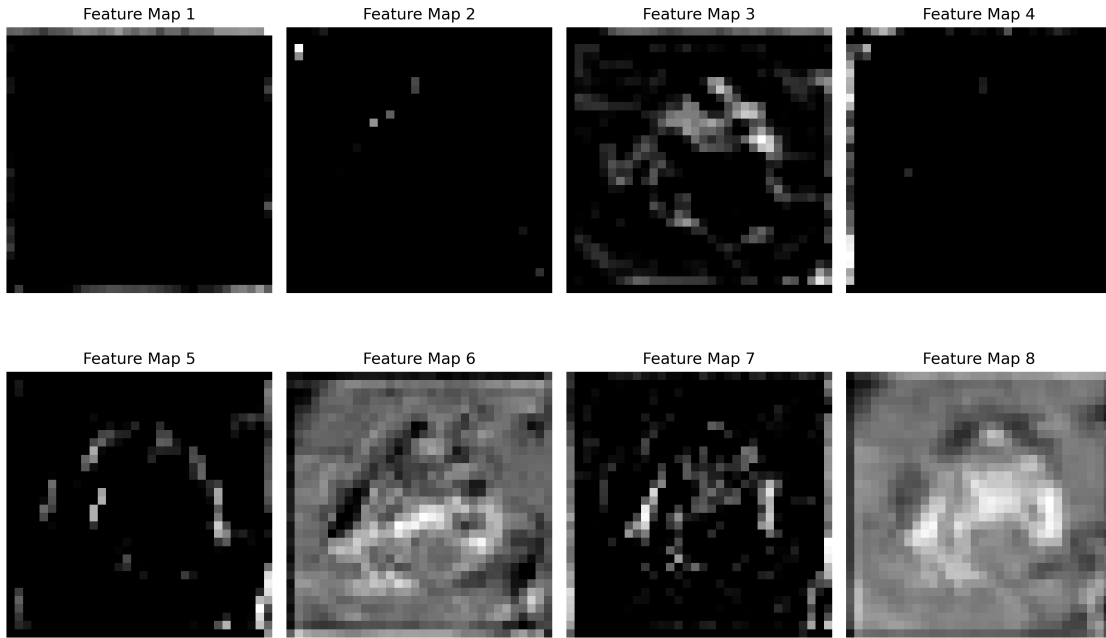


Figure 5: Feature maps generated by the first convolution layer

Inference:

Each feature map represents the response of a convolution filter to different visual patterns in the input image. Different filters can detect edges, textures, and other local characteristics.

6.5 Task 5: Max Pooling and Average Pooling

A 2×2 pooling window with stride 2 was used to compare max pooling and average pooling.

Table 4: Pooling output dimensions

Operation	Output Shape
Original Feature Map	$1 \times 32 \times 32 \times 16$
Max Pooling	$1 \times 16 \times 16 \times 16$
Average Pooling	$1 \times 16 \times 16 \times 16$

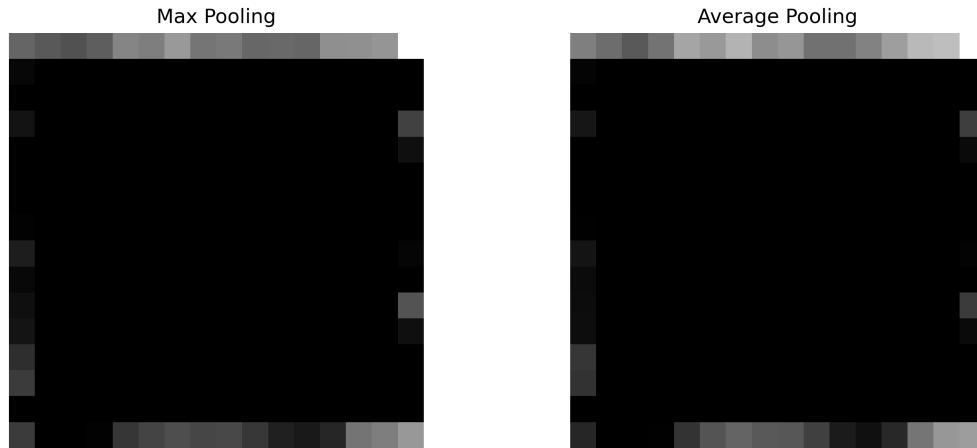


Figure 6: Comparison of max pooling and average pooling

Inference:

Both pooling operations reduce the spatial dimensions by a factor of two. Max pooling retains the strongest activation in each window, whereas average pooling calculates the mean activation.

6.6 Task 6: CNN Construction and Training

The CNN architecture used in this experiment is:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

The model was trained using the following parameters:

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32
- Loss Function: Sparse Categorical Crossentropy

The model contains:

545098

trainable parameters.

7 Training Results

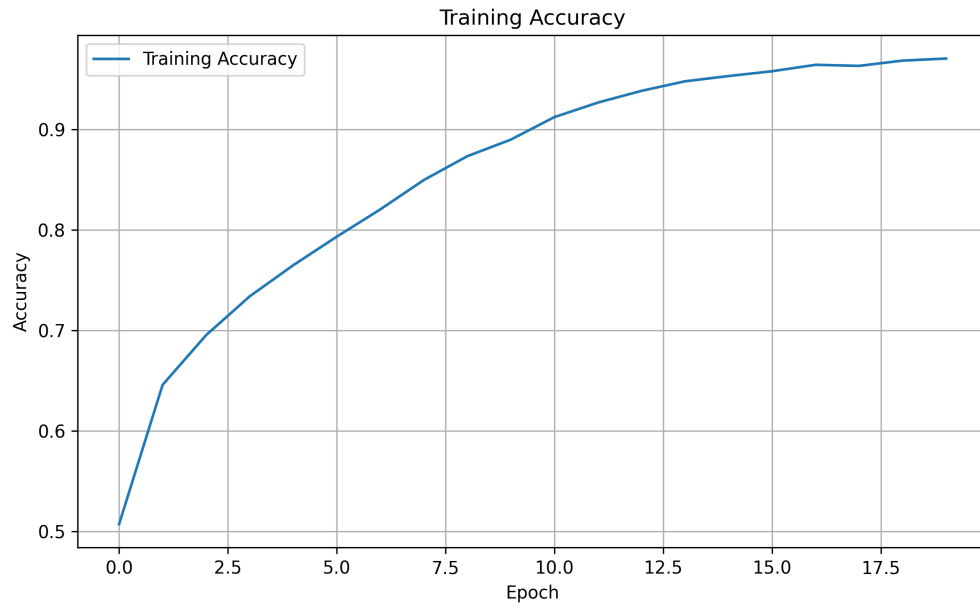


Figure 7: Training accuracy

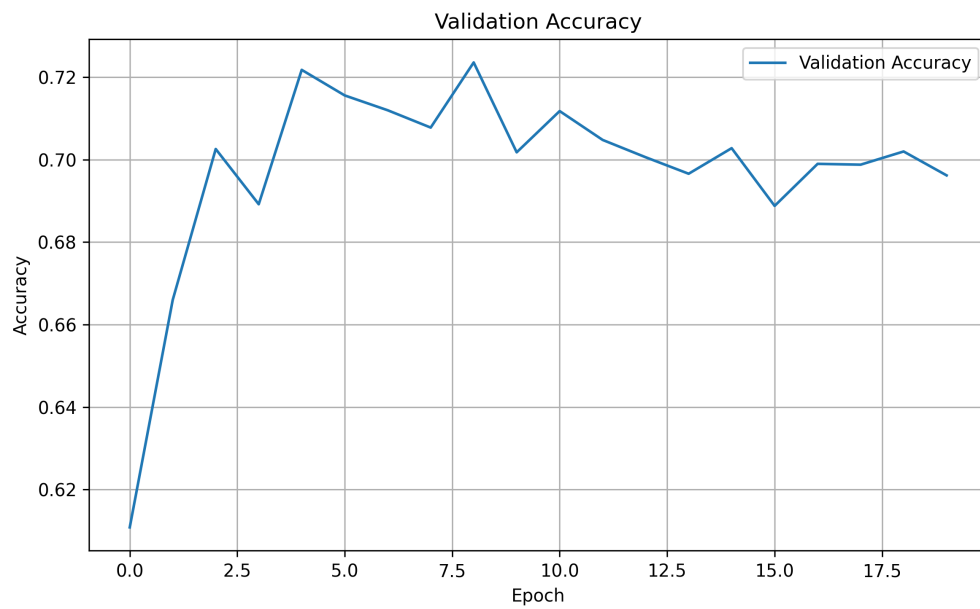


Figure 8: Validation accuracy

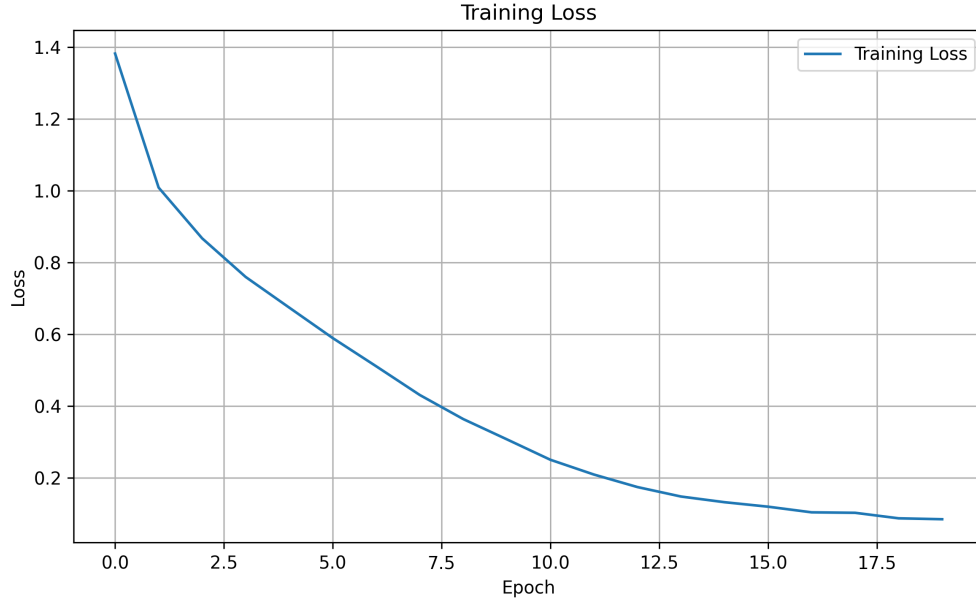


Figure 9: Training loss

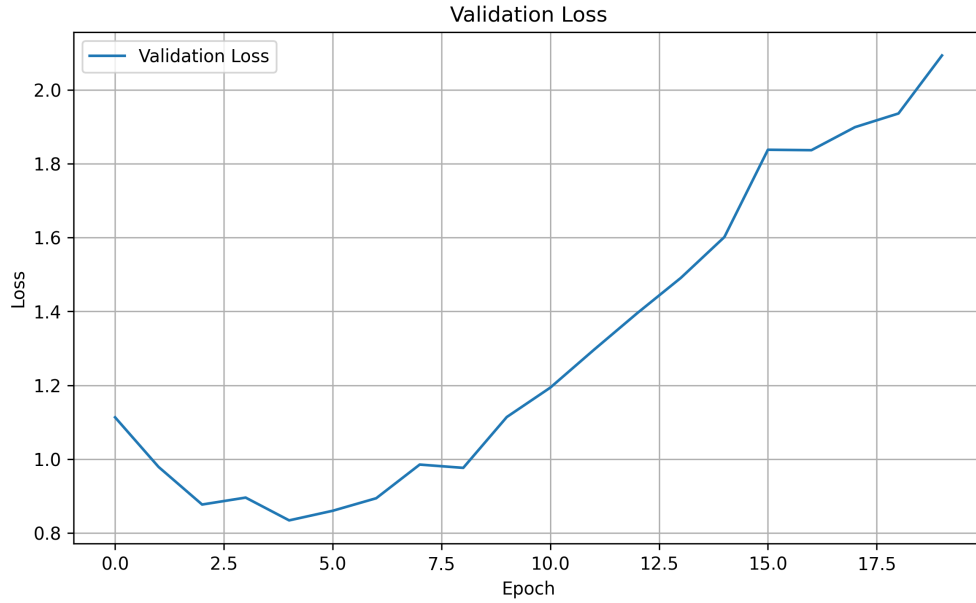


Figure 10: Validation loss

Inference:

The training accuracy increased substantially during the 20 epochs and reached 97.06%. Validation accuracy reached 69.62%. The gap between training and validation accuracy indicates that the model has some degree of overfitting.

8 Model Evaluation

The trained CNN was evaluated using the CIFAR-10 test dataset.

Table 5: Final CNN performance

Metric	Value
Training Accuracy	97.06%
Validation Accuracy	69.62%
Testing Accuracy	68.04%
Precision	69.07%
Recall	68.04%
F1-score	68.28%
Test Loss	2.2466
Trainable Parameters	545,098

8.1 Classification Report

Table 6: Class-wise classification performance

Class	Precision	Recall	F1-score
Airplane	0.75	0.70	0.73
Automobile	0.80	0.77	0.79
Bird	0.64	0.52	0.57
Cat	0.46	0.50	0.48
Deer	0.61	0.62	0.62
Dog	0.51	0.70	0.59
Frog	0.74	0.75	0.74
Horse	0.79	0.71	0.74
Ship	0.81	0.82	0.81
Truck	0.80	0.72	0.75

8.2 Confusion Matrix

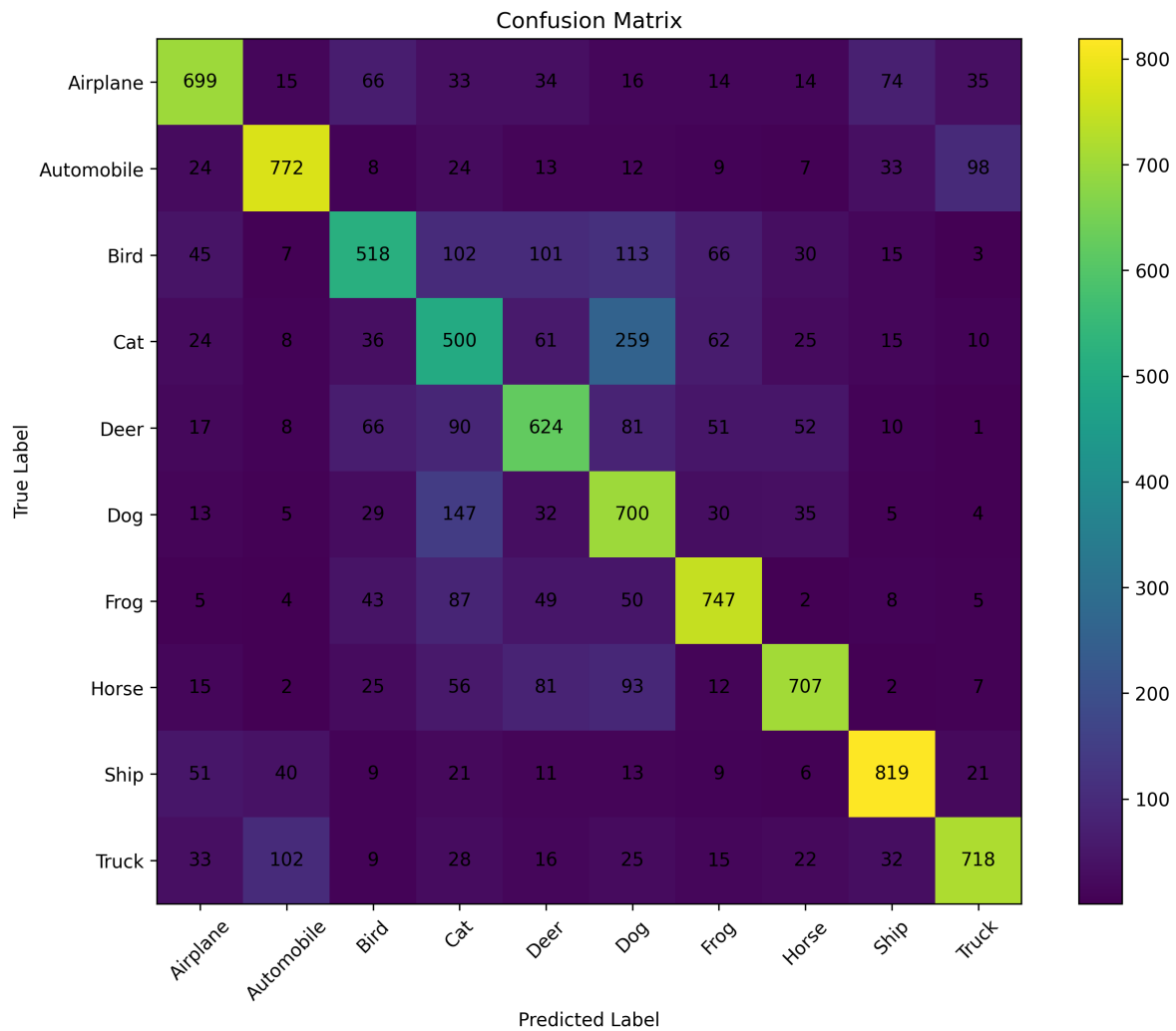


Figure 11: Confusion matrix for CIFAR-10 classification

Inference:

The confusion matrix shows that the model performs relatively well on classes such as Ship and Automobile. More confusion is observed among visually similar classes such as Cat, Dog, Bird, and Deer.

9 Additional Exercises

9.1 1. Output Size Calculation

For a 64×64 image with a 5×5 kernel, stride 2, and padding 2:

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 \quad (6)$$

$$= \left\lfloor \frac{63}{2} \right\rfloor + 1 \quad (7)$$

$$= 32 \quad (8)$$

Therefore:

$$\boxed{32 \times 32}$$

9.2 2. Trainable Parameters

For 64 filters of size 3×3 and an RGB input:

$$P = (3 \times 3 \times 3 + 1) \times 64 \quad (9)$$

$$= 28 \times 64 \quad (10)$$

$$= \boxed{1792} \quad (11)$$

9.3 3. ReLU and Sigmoid

ReLU is defined as:

$$f(x) = \max(0, x)$$

ReLU is computationally simple and is commonly used in hidden layers of CNNs.

The sigmoid function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

It maps values to the range $(0, 1)$ and is useful when an output represents a probability. However, sigmoid can suffer from vanishing gradients for very large positive or negative inputs.

9.4 4. Max Pooling and Average Pooling

Max pooling selects the maximum activation from each pooling window, preserving the strongest detected feature.

Average pooling calculates the mean activation in each window, producing a smoother representation.

In this experiment, both operations reduced a 32×32 feature map to 16×16 .

9.5 5. Increasing the Number of Filters

Increasing the number of convolution filters from 16 to 64 allows the network to learn more feature representations.

However, this also increases the number of trainable parameters and computational requirements. The effect on classification accuracy and computation time should be evaluated experimentally.

10 Discussion

10.1 1. Why is convolution preferred over fully connected layers for images?

Convolution exploits the spatial structure of images by using local receptive fields and shared weights. This reduces the number of parameters compared with fully connected layers.

10.2 2. How does stride affect the feature map size?

Increasing the stride causes the convolution window to move in larger steps across the input. Therefore, the resulting feature map has smaller spatial dimensions.

10.3 3. What is the role of padding?

Padding adds pixels around the boundary of the input image. Same padding can preserve spatial dimensions, while valid padding does not add padding and therefore generally reduces the output size.

10.4 4. Why is pooling used?

Pooling reduces spatial dimensions and computational requirements while retaining important information from feature maps. It also provides a degree of spatial invariance.

10.5 5. How do feature maps represent image characteristics?

Feature maps contain the responses produced by convolution filters. Different filters can respond to different local patterns such as edges, textures, shapes, and other visual characteristics.

10.6 6. Why do CNNs require fewer parameters than MLPs?

CNNs use local connectivity and weight sharing. The same convolution filter is applied across different spatial locations, so the network does not require separate weights for every input-to-neuron connection.

11 Result

A CNN was successfully implemented for CIFAR-10 image classification.

The final model achieved:

- Training Accuracy: 97.06%
- Validation Accuracy: 69.62%
- Testing Accuracy: 68.04%
- Precision: 69.07%
- Recall: 68.04%
- F1-score: 68.28%
- Number of Parameters: 545,098

12 Conclusion

The experiment demonstrated the implementation of a Convolutional Neural Network for image classification using TensorFlow/Keras. Convolution with different kernel sizes, stride and padding, feature-map visualization, max pooling, average pooling, and CNN classification were studied.

The CNN achieved a testing accuracy of 68.04% on the CIFAR-10 test dataset. The results demonstrate the ability of CNNs to automatically learn useful visual features for image classification.

13 References

1. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*.
2. Bishop, C. M., *Pattern Recognition and Machine Learning*.
3. Haykin, S., *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.